AI ASSISTED DEVELOPMENT PROGRAM

Day 4

Agent Harness, Agent Orchestration & Economics of AI Development

By Shivam Patel · LinkedIn

Shivam Patel

Sr. ML Engineer @ Apple | Retrieval, RAG & Agentic Workflows | Ex-Google · Ex-Adobe | CMU

Technical Specializations

- RAG & Retrieval for LLMs
- NLP, Transformers & Agentic AI
- Multi-Modal Deep Learning (Vision + Language)
- Large-Scale Distributed Training & Inference
- LLM Evaluation, Alignment
- Production ML Systems - Siri, Search, Feature Engineering

🔹 Career Highlights

- At Apple, **improving retrieval & ranking models for RAG systems** and agentic workflows powering Siri & LLMs
- Built large-scale recommendation systems, neural architecture search, privacy-preserving ML, LLM-based feature engineering at **Google**
- Developed transformer-based multi-modal architectures for text-based video object segmentation 2 US Patents - text-based video object selection & AI-blockchain recruitment at **Adobe**

🎓 Academic & Teaching

- **MS, Computer Science - Carnegie Mellon University**
- **Research collaborations: MIT, Caltech, Stanford, Harvard**
- **Erdos Number 2 · Microsoft Global Hackathon Winner**
- **Instructor @ Acceler** - ML, DL, GenAI, production Python
- Trained **3000+ working professionals** in ML, GenAI & Agentic AI Visiting
- Researcher at Mila under Turing Laureate Prof Yoshua Bengio Visiting
- Research Student at Cambridge (Centre for Existential Risk)

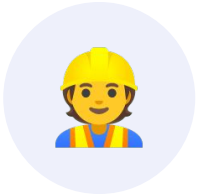
acceler

WELCOME TO TODAY’S CLASS! BEFORE WE BEGIN,

Pop Into The Chat

Your Name

Introduce yourself to the room



Your Role

What you do day-to-day

Your Location

Where you’re joining from

GET THE MOST OUT OF IT

Optimise Your Experience

01

Be Vocal

Don’t be shy to speak up and get clarifications!

02

Be Confident

Use your resources, solve assignments, MCQs & assessments. What you put in is what you’ll get out.

03

Be Active

Interact with your instructors via the live class.

SET YOURSELF UP TO WIN

Learn These Success Hacks

- ✓ Attend every session
- ✓ Be attentive in the class
- ✓ Participate in class activities and demos
- ✓ Actively advocate for yourself
- ✓ Raise post-class help on the WhatsApp Group
- ✓ Ensure you have WhatsApp Group access
- ✓ Consistency is the key
- ✓ Be patient with yourself!

Don’t worry!

We are here to help at your every step!

SETUP

Virtual Lab Access

Before we dive in, let’s take a quick moment to ensure everyone is set up properly. Your Virtual Lab access is essential for hands-on work during this session.

LET’S DO A QUICK WALKTHROUGH TOGETHER

Virtual Lab Access & Login

✓

Is your Nuvepro VM running?

Confirm your virtual machine has started and is responsive.

✓

Signed in on Chrome?

Use the provided Email credentials to sign in on Chrome.

✓

Can you access the lab tools?

Open the tools inside the lab environment and verify access.

ENVIRONMENT

Setting Up Claude Code

Get Claude Code running in your lab environment — the AI coding agent we'll use all day.

ROADMAP

Agenda For Today

Agent Harness

The model plus everything you build around it — and why it matters.

Agent Orchestration

From one agent to a fleet — subagents, agent teams & workflows.

Economies of AI Development

CapEx, OpEx and the total cost of ownership of AI-assisted work.

Summary

Summary of what we have learnt so far

PART 1

Agent Harness

A coding agent is the model plus everything you build around it.

DEFINITION

What Is a Harness, Really?

- **A coding agent is the model plus everything you build around it.** The harness is that scaffolding — prompts, tools, infrastructure and control logic.
- **Harness engineering treats that scaffolding as a real artifact** — designed, versioned and reviewed like any other system.
- **And it tightens every time the agent slips** — each failure becomes a new rule, guardrail or check.

THE BIG IDEA

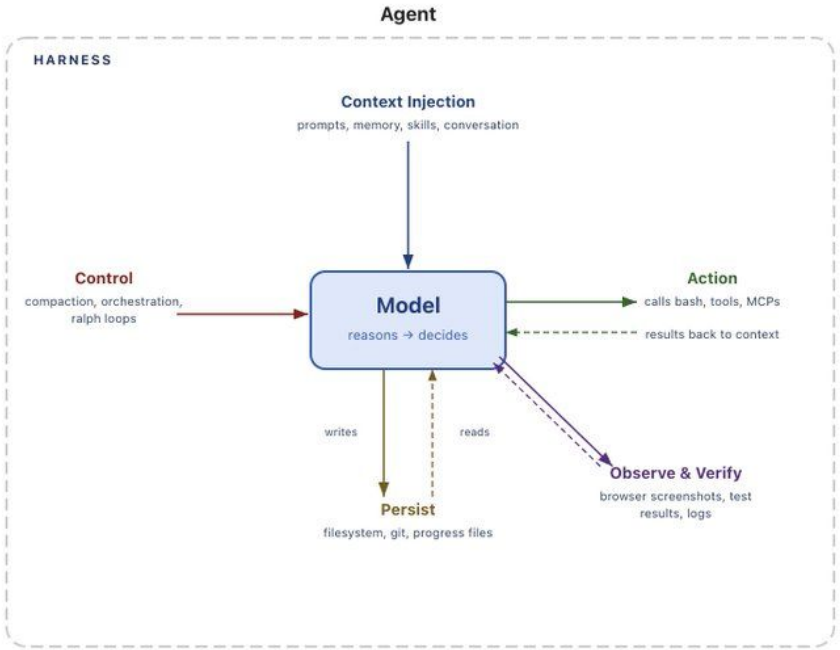
Agent = Model + Harness

If you're not the model,

you're the harness.

THE ANATOMY

Agent Harness



THE ANATOMY

What the Harness Includes

Prompts & agent files

System prompts, CLAUDE.md, AGENTS.md, skill files, and subagent prompts.

Tools & MCP servers

Tools, skills, MCP servers — and their descriptions.

Bundled infrastructure

Filesystem, sandbox, and browser the agent works inside.

Orchestration logic

Subagent spawning, handoffs, and model routing.

Hooks & middleware

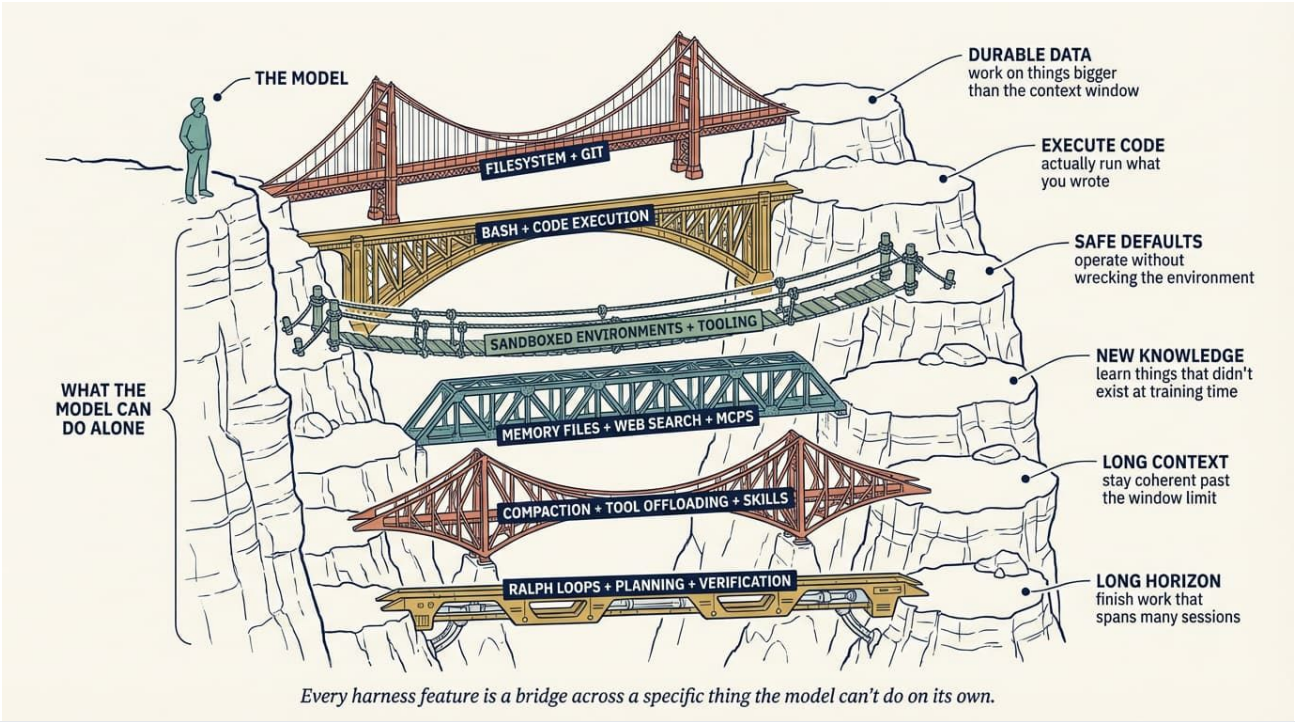
Deterministic execution: compaction, continuation, lint checks.

Observability

Logs, traces, cost and latency metering.

THE ANATOMY

Agent Harness



Every harness feature is a bridge across a specific thing the model can't do on its own.

FROM A MODEL'S PERSPECTIVE

Why Do We Need Harnesses?

Models (mostly) take in text, images, audio and video — and output text. That's it. Out of the box they cannot:

💾 Maintain durable state

No memory across interactions — the harness carries state forward.

💻 Execute code

Models output text; they can't run what they write on their own.

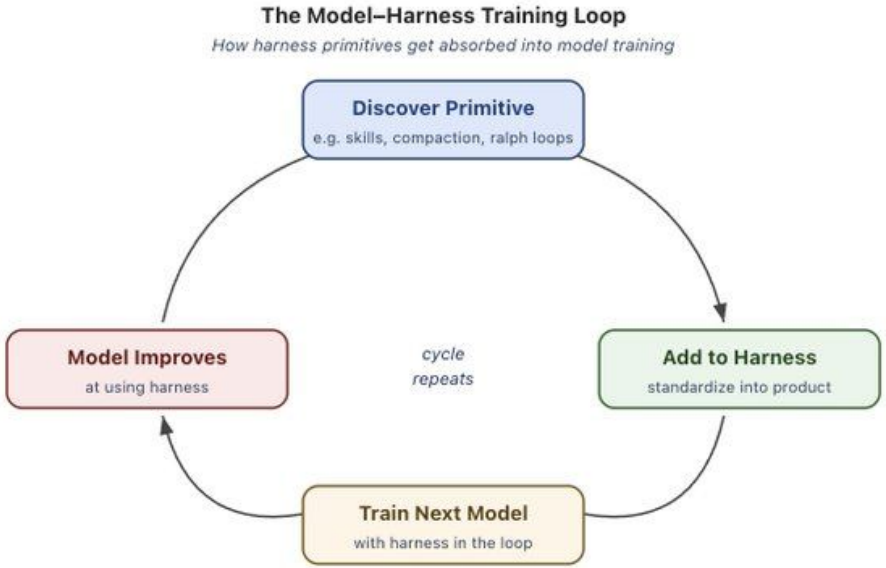
🌐 Access realtime knowledge

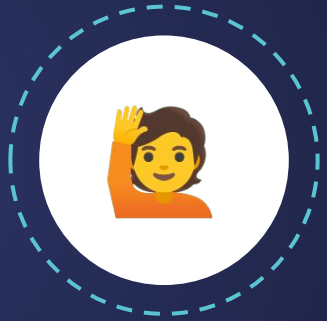
No live view of the world — tools fetch what training data can't.

📦 Set up environments

Installing packages and preparing environments to complete work.

The Model–Harness Training Loop





QUICK CHECK WITH THE LEARNERS

Agent, Model & Harness

Who can explain the difference between an agent, a model and a harness? Drop it in the chat.

THE ANSWER

Agent, Model and Harness: What's the Difference?

Component	What it does	Plain-language analogy
Model	Reasons, predicts and generates text or other outputs	The "brain" of the system
Harness	Executes actions, manages memory, runs tools and enforces rules	The "body" and workspace around the brain
Agent	The full working system that combines the two	A worker who can think and act

THE DISCIPLINE

Harness Engineering

Discipline	What it focuses on	Main artifact	Typical applications
Prompt engineering	Wording the input to get a better response	A well-crafted prompt	Early LLM applications
Context engineering	Curating what information the model sees and when	Retrieval pipelines, memory design	RAG-era applications
Harness engineering	Designing the full system around the model — tools, sandboxes, loops, guardrails	The harness itself	Agentic systems and autonomous workflows

FIELD NOTES

Common Failure Modes in Production Agent Harnesses

Most operational agent failures come from the harness, not the model itself. These are the most common problems teams hit in real-world systems:

- ✗ **Context rot**
- ✗ **Tool overload**
- ✗ **Brittle tool wiring**
- ✗ **Latency**
- ✗ **Irrelevant material**
- ✗ **Weak verification**
- ✗ **Missing guardrails**

THE BLUEPRINT

Core Components of an Effective Harness

1. Context Engineering & Management
2. Tool Orchestration and Guardrails
3. Human-in-the-Loop (HITL) Controls
4. Lifecycle and State Management

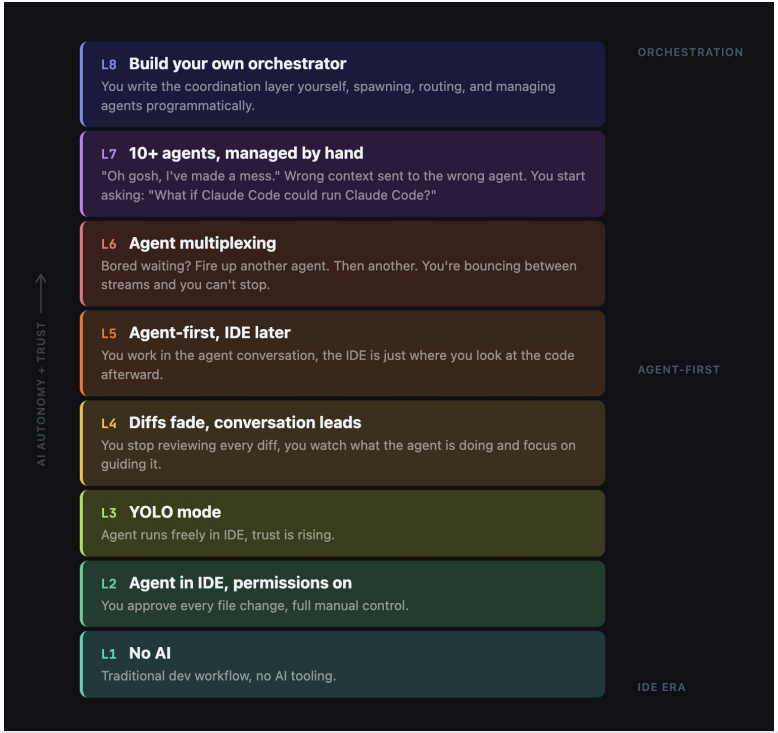
PART 2

Agent Orchestration

A conductor runs one agent. An orchestrator runs the fleet.

THE LADDER

8 Levels of AI-Assisted Coding



THE TWO MODES

Conductor vs. Orchestrator

CONDUCTOR

One agent, synchronous

- You steer every step and review every diff — in the loop the whole time.
- Precise, powerful, one-at-a-time.
- Pair programming, leveled up.

ORCHESTRATOR

Many agents, async

- You write the brief; they clone the repo, branch, code, test, and hand back a PR.
- Your effort front-loads onto the spec and back-loads onto the review.
- The middle runs itself.

Remember A conductor runs one agent. An orchestrator runs the fleet.

THE SHIFT

From Conductor to Orchestrator

- **The role is moving:** coder → conductor (guide one agent) → orchestrator (direct many).
- **The mental model shift** is pair programming vs. managing a team. In the conductor model you guide a single agent in real time — synchronous, sequential, and your context window is a hard ceiling.
- **Tools for this model** include Claude Code CLI and Cursor's in-editor agent mode.
- **Like managing a real team,** you need different skills now: clear specs, work decomposition, and output verification rather than writing code yourself.

~90%

of engineers already use AI to write code. The frontier isn't one assistant anymore — it's a fleet.

THE CEILING

Drawbacks of a Single Agent

✗

Context overload

✗

No specialization

✗

No coordination

THE PAYOFF

Why Multi-Agent?

✓ Parallelism

Many agents make progress on independent pieces at the same time.

✓ Specialization

Focused context — each agent carries only what its job needs.

✓ Isolation

Safe execution — agents work in their own space without side effects.

✓ Compound learning

What one run teaches you gets encoded and reused by every run after it.

PATTERN 1

SubAgents — Focused Delegation

Already introduced in a previous session — we'll quickly go through it in the code demo.

PATTERN 2

Agent Teams — True Parallel Execution

We will focus more on this pattern in today’s session.

SWEET SPOTS

When to Use Agent Teams

Agent teams are most effective for tasks where parallel exploration adds real value.

Research and review

Teammates investigate different aspects simultaneously, then share and challenge each other's findings.

New modules or features

Teammates each own a separate piece without stepping on each other.

Debugging with competing hypotheses

Teammates test different theories in parallel and converge on the answer faster.

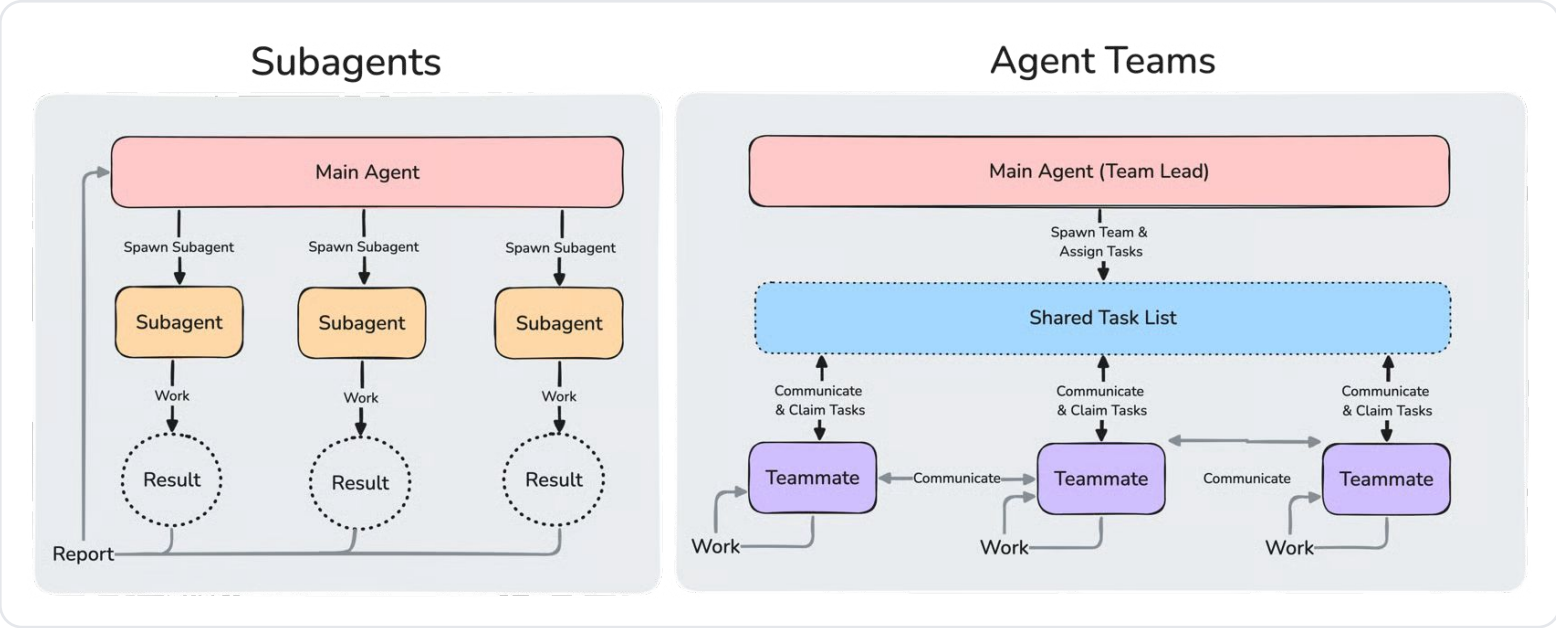
Cross-layer coordination

Changes spanning frontend, backend, and tests — each owned by a different teammate.

Caveat For sequential tasks, same-file edits, or work with many dependencies, a single session or subagents are more effective.

SIDE BY SIDE

Comparison With SubAgents



SIDE BY SIDE

Comparison With SubAgents

	Subagents	Agent teams
Context	Own context window; results return to the caller	Own context window; fully independent
Communication	Report results back to the main agent only	Teammates message each other directly
Coordination	Main agent manages all work	Shared task list with self-coordination
Best for	Focused tasks where only the result matters	Complex work requiring discussion and collaboration
Token costs	Lower — results summarized back to main context	Higher — each teammate is a separate Claude instance

HANDS-ON

Code Demo

Let’s switch to the lab — subagents and agent teams in action.

GOING FURTHER

Dynamic Workflows

Dynamic workflows orchestrate many subagents from a script Claude writes and you can rerun. Use them for:

Codebase audits

Fan out subagents across the repo and aggregate their findings.

Large migrations

Repeatable, scripted changes applied across many files or services.

Cross-checked research

Independent agents verify each other's answers before results land.

DECISION GUIDE

When to Use Dynamic Workflows

Subagents, skills, agent teams, and workflows can all run a multi-step task. The difference is who holds the plan:

	Subagents	Skills	Agent teams	Workflows
What it is	A worker Claude spawns	Instructions Claude follows	A lead agent supervising peer sessions	A script the runtime executes
Who decides what runs next	Claude, turn by turn	Claude, following the prompt	The lead agent, turn by turn	The script
Where intermediate results live	Claude's context window	Claude's context window	A shared task list	Script variables
What's repeatable	The worker definition	The instructions	The team definition	The orchestration itself
Scale	A few delegated tasks per turn	Same as subagents	A handful of long-running peers	Dozens to hundreds of agents per run
Interruption	Restarts the turn	Restarts the turn	Teammates keep running	Resumable in the same session

ECOSYSTEM

Plugins in Claude Code

Discover ready-made plugins that extend Claude Code with new skills, tools and workflows.

code.claude.com/docs/en/discover-plugins

HANDS-ON

Code Demo

Dynamic workflows — live in the lab.

ADVANCED PATTERN

The Ralph Loop and Self-Improving Agents

The Ralph Loop breaks development into small atomic tasks and runs an agent in a stateless-but-iterative loop — avoiding context overflow while maintaining continuity through external memory.

1

Pick

Select the next task from tasks.json.

2

Implement

Make the change.

3

Validate

Run tests, types, lint.

4

Commit

If checks pass, commit and update task status.

5

Reset

Clear the agent context and start fresh with the next task.

COMPOUND LEARNING

Making Agents Smarter Over Time

Self-reflection

After every task, force the agent to write a REFLECTION.md.

Token budgeting & kill criteria

Set hard per-agent budgets — and stop runs that blow past them.

Beads / persistent memory

Gastown's "beads" pattern: immutable, git-backed records of every decision and outcome with full provenance.

THE MENTAL MODEL

The Factory Model

You’re no longer just writing code. You’re building the factory that builds your software. That factory has a production line:

1 Plan

Write specs with acceptance criteria. Your spec is the leverage.

2 Spawn

Create your team and assign agents.

3 Monitor

Watch progress and resolve blockers every 5–10 minutes. Don’t hover.

4 Verify

Run tests, review code. Verification is the bottleneck, not generation.

5 Integrate

Merge branches, resolve conflicts.

6 Retro

Update AGENTS.md with new patterns. Compound learning.

OPERATIONS

Practical Tips for Running the Factory

1 Set WIP limits

Don’t run more agents than you can meaningfully review. 3–5 is the sweet spot.

2 Define kill criteria

If an agent is stuck for 3+ iterations on the same error, stop and reassign.

3 Async check-ins

Check progress every 5–10 minutes. Let agents work autonomously.

4 One file, one owner

Never let two agents edit the same file. Conflicts kill velocity.

TAKE ACTION

5 Patterns to Start Today

1. **Subagents for decomposition**
   Use the Task tool to spawn focused child agents with specific briefs and file ownership. Zero setup. Start here today.

2. **Agent Teams for parallelism**
   Enable CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1. Create a lead + 3 teammates. Use the shared task list for coordination.

3. **Git worktrees for isolation**
   Each agent gets its own worktree. No merge conflicts, clean integration. Tools like Conductor handle this automatically.

4. **Quality gates for trust**
   Require plan approval for risky changes. Add hooks that run tests on task completion. Never trust agent output without verification.

5. **AGENTS.md for compound learning**
   Document patterns, gotchas, and style preferences. Every session reads it, every session updates it. Knowledge compounds.

REALITY CHECK

Challenges and the Human Role in Orchestration

Quality control & trust

Coordination & conflict

Context, shared state & hand-off

Prompting and specifications

Tooling and debugging

Ethics and responsibility

PART 3

Economies of AI Development

Shipping fast is the cheap part — running it, fixing it, and keeping it alive is where the money goes.

THE REFRAME

Speed Was Never the Expensive Part

- **The entire AI pitch is velocity** — write more code, faster.
- **But shipping fast is the cheap part.** Running it, fixing it, and keeping it alive is where the money actually goes.
- **So the number that matters isn't velocity.** It's the full cost of owning the software over its life.

THE NUMBER THAT MATTERS

Total Cost of Ownership

Not how fast you shipped —
what it costs to keep it alive.

THE FRAMEWORK

The Cost Model

CAPEX

The upfront cost to build

- One-time investment made before the software exists.
- Design, architecture, tests, scaffolding.

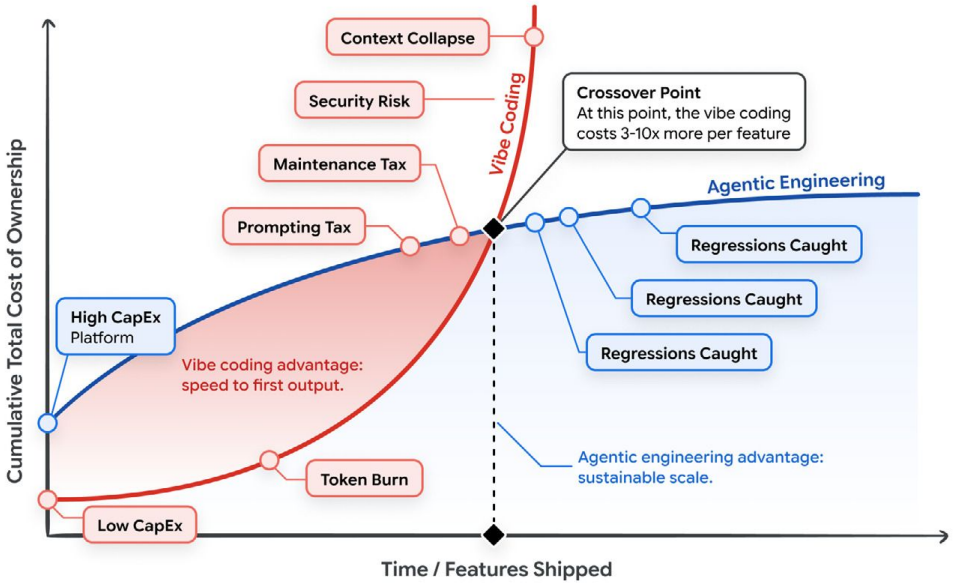
OPEX

The ongoing cost to run

- Running, fixing, and maintaining the system.
- It never stops.

The shift nobody budgeted for In AI-assisted work, OpEx is now a token bill.

The Economies of AI Development



Vibe Coding

12%

CapEx
Min Investment

OpEx
High Running Costs

- Rapid prototyping, slow scaling
- High friction for long-term maintenance
- Economic dead-end for complex systems

Agentic Engineering

12%

CapEx
Upfront Platform Design

OpEx
Low Marginal Running Costs

- Controlled iteration, fast scaling
- Low friction for automatic updates
- Economically sustainable for mature codebases

LOW CAPEX · HIGH OPEX

The Hidden Debt of Vibe Coding

CapEx is basically zero — a monthly subscription and a few prompts. No system design, no upfront investment. That's the whole appeal. Then the OpEx compounds:

Token burn

Dumping huge files into context and asking the model to fix its own mistakes creates a prompting loop. Low first-pass success — and every retry burns tokens.

Maintenance tax

Ad-hoc prompts produce code with no structure. Six months on, a real engineer spends days reverse-engineering AI spaghetti to fix one bug.

Security remediation

Fast code generation means fast vulnerability generation — and a flaw caught in production costs exponentially more than one caught at design.

HIGH CAPEX · LOW OPEX

The Investment of Agentic Engineering

Agentic engineering flips the model — real engineering time goes in before a single line of production code exists. The CapEx:

📐 Design the API schemas

Define the contract before the code.

✅ Build deterministic test suites

The output gets checked, not trusted.

🎯 Structure the agent's context

The one that matters most — govern what the AI sees and it can't wander.

The payoff The marginal cost of every feature after that drops off a cliff — the AI works inside a governed factory: structurally sound, pre-tested, aligned with your standards.

TOKEN ECONOMICS

Scaling Efficiency via Dynamic Context and Skills

- **LLMs charge for every token you send.** Passing a 100,000-token repo into every prompt isn’t a style choice — at scale it’s just unviable.
- **The fix is a financial strategy that happens to look technical:** send the model dense, high-signal context, not a noisy dump.

REMEMBER THIS

Context isn’t a technical detail.

It’s a line item.

HANDS-ON

Cost Engineering Playbook

AI Cost Playbook — Six patterns that cut your LLM API bill by 60–80% without removing a single feature — real code, real cost math, eight modules.

Learner Link

Scaling Efficiency: 3 Levers

1

Lever 1 — Tight payload (the big one)

Feed the agent a precise AGENTS.md and architectural guardrails instead of the whole sprawling repo. Right context upfront means a higher first-pass success rate — the exact retry loop vibe coding can't escape.

2

Lever 2 — Dynamic context

Skills & tool calling — agents pull context dynamically instead of carrying it all. Think Model Context Protocol servers. (Full breakdown in the day-3 paper.)

3

Lever 3 — Intelligent model routing

Big models for the hard thinking — requirements, architecture, first implementation. Small, fast, cheap models for deterministic grunt work — tests, review, CI/CD monitoring. Same output quality, a fraction of the token cost.

KEY TAKEAWAY

Scaling Efficiency via Dynamic Context and Skills

To truly optimize OpEx, advanced agentic engineering relies on dynamic context through the use of “skills” or tool calling (such as Model Context Protocol servers) — which we cover in detail in the day-3 paper.

LEVER 3

Intelligent Model Routing

Stop paying frontier prices to fix a typo. Vibe coding runs every request through one massive frontier model — premium token rates for a typo fix or a basic unit test.

HEAVY MODELS

Hard problems

- Requirements, architecture, initial implementation.
- The work that actually needs the horsepower.

CHEAP MODELS

Routine work

- Test generation, code review, CI/CD monitoring.
- Deterministic, low-stakes — flows to smaller, faster, far cheaper models automatically.

The payoff Same output quality. A fraction of the token cost.

HANDS-ON

Code Demo

Supermemory — Harness hooks + Compound Learning

THE PRINCIPLE

AI Amplifies Whatever Culture It Lands In

Individual builder or org leader — same principle: AI doesn't fix your engineering culture, it scales it. Both directions. What follows turns that into moves you can make now.

For Individual Builders

Building one agent beats reading about a hundred. The three moves:

1 Write your AGENTS.md first

Ten lines: stack, conventions, hard rules, workflow. Then add a rule every single time the agent does something it shouldn’t do again. It grows itself.

2 Tests and evals before the code

That suite is your contract with the AI — it states intent more precisely than any prompt can. This is what turns vibe coding into agentic engineering.

3 Make one workflow your first agent

A recurring report, a review process, a research loop. Prototype it, graduate it when it earns its keep. Building one end to end teaches more than reading about a hundred.

Don’t go soft Review every line that ships, distrust anything that looks clever, and keep your own debugging and design instincts sharp. AI scales your expertise — it doesn’t replace it.

For Engineering Leaders

Set the bar at the eval, not the demo.

1 The eval is the bar

A demo proves an agent can succeed once. A passing eval with a real rubric proves it succeeds reliably. Gate shipping on eval coverage the way you gate deployment on test coverage.

2 Prototype ≠ production

Vibe coding is the right speed for exploring; agentic engineering is the right discipline for shipping. Keep the boundary blurry and you get prototypes that ship by accident.

3 The harness is infrastructure

AGENTS.md, system prompts, eval suites, skill libraries — reviewed in PRs, versioned, owned by named engineers. Build it once, refine it many times.

For Organizations

Speed without scaffolding is just faster debt.

1 Fund it as engineering investment

The biggest gains come from pairing AI tooling with eval coverage, observability, and architectural standards. Skip the scaffolding and you buy speed without quality — debt that compounds.

2 Build the substrate before you scale

Evals in CI, a trace of every agent run, scoped permissions per agent, security review tuned to how generated code fails. A prototype on a laptop is not a production system.

3 Hire and grow for judgment

As writing code gets automated, the bottleneck shifts to specification, evaluation, and architectural judgment. The most valuable engineers will be the ones who direct agents best.

Lock in early Open standards — MCP for tools, A2A for cross-agent — and plan for hybrid teams: humans set direction, agents implement across a clear handoff.

HANDS-ON

Code Demo

Token Budget Lab at Scale - Building At Scale

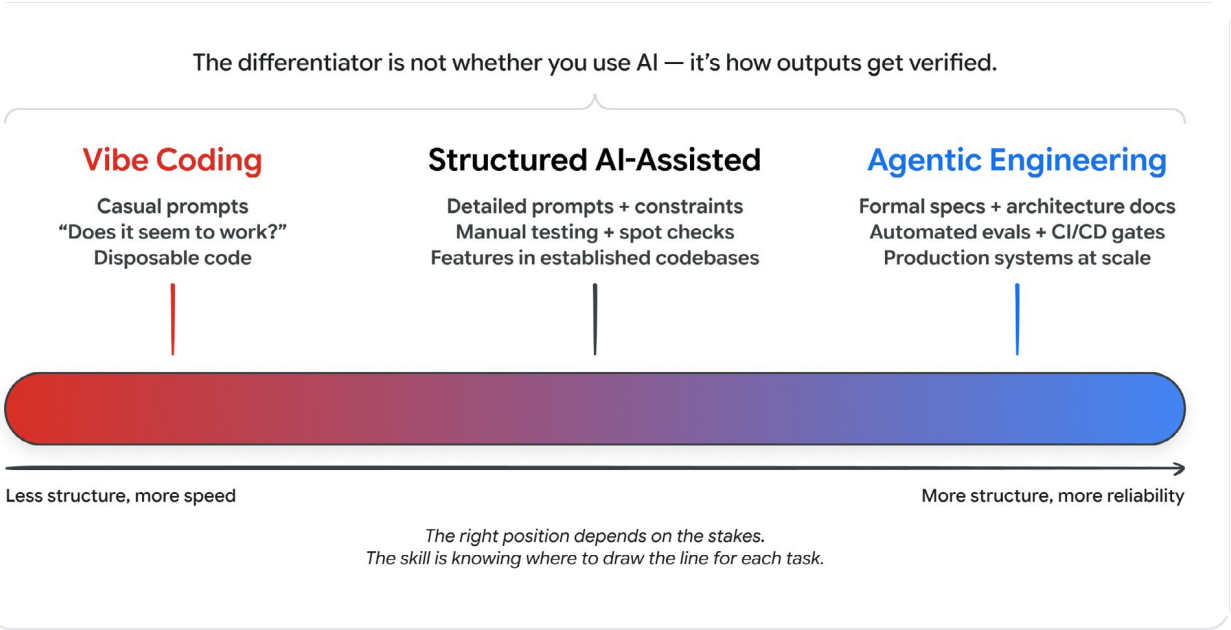
PART 4

Summary

Recap of what we have learnt so far

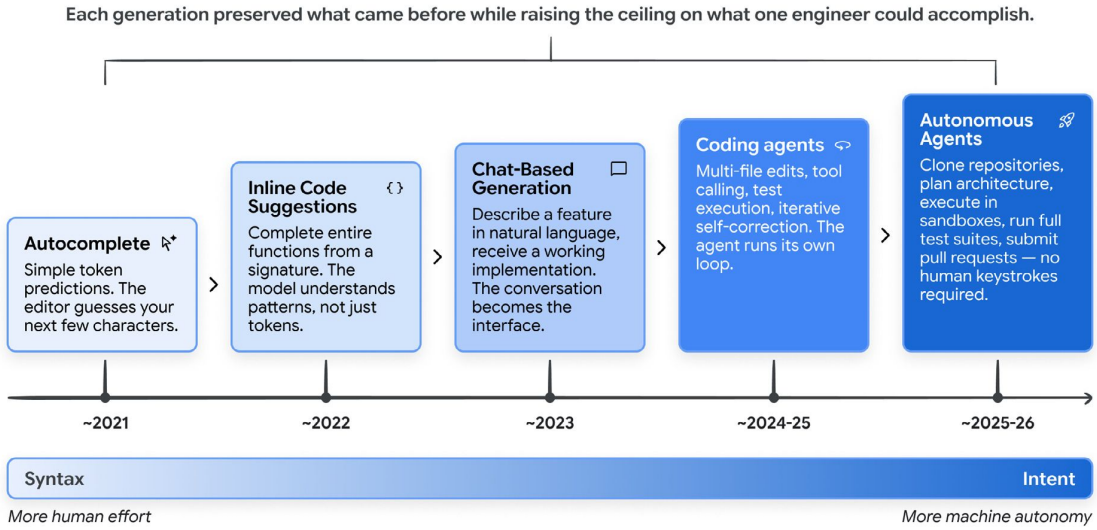
RECAP

The Line Between Vibe Coding and Engineering



RECAP

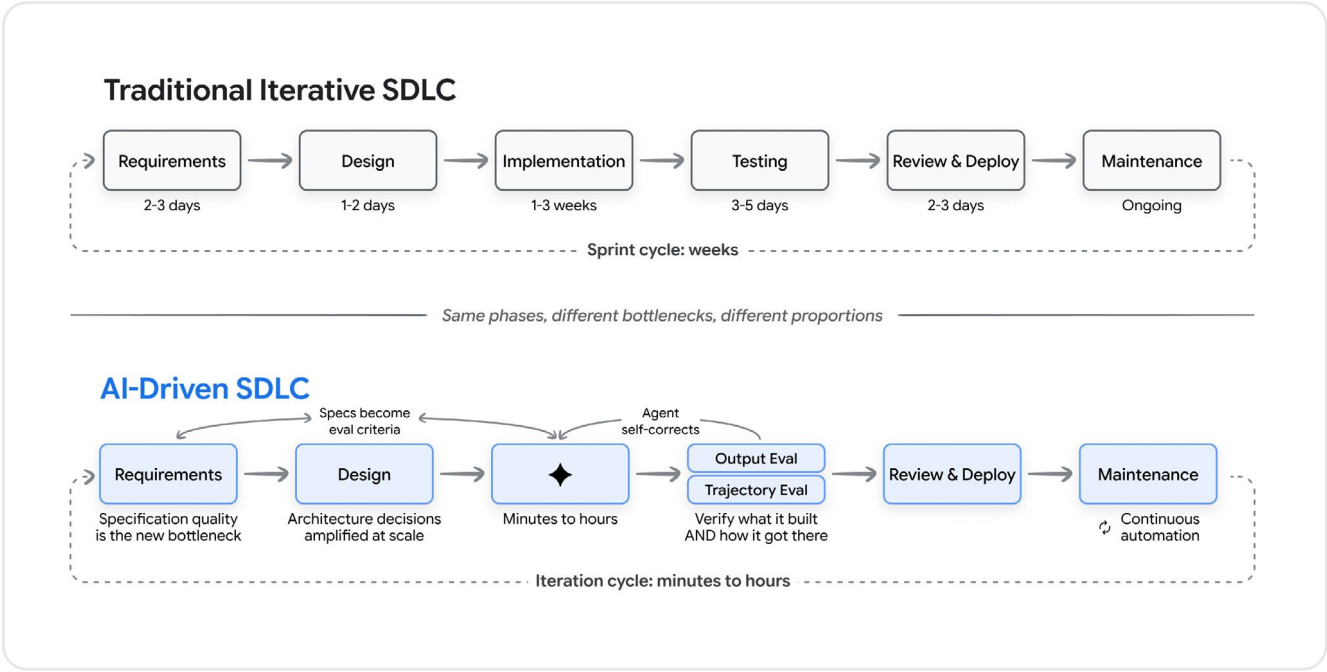
Where We Stand Today



85% of developers now use AI coding tools regularly.
41% of new code is AI-generated.
— 2026 industry data

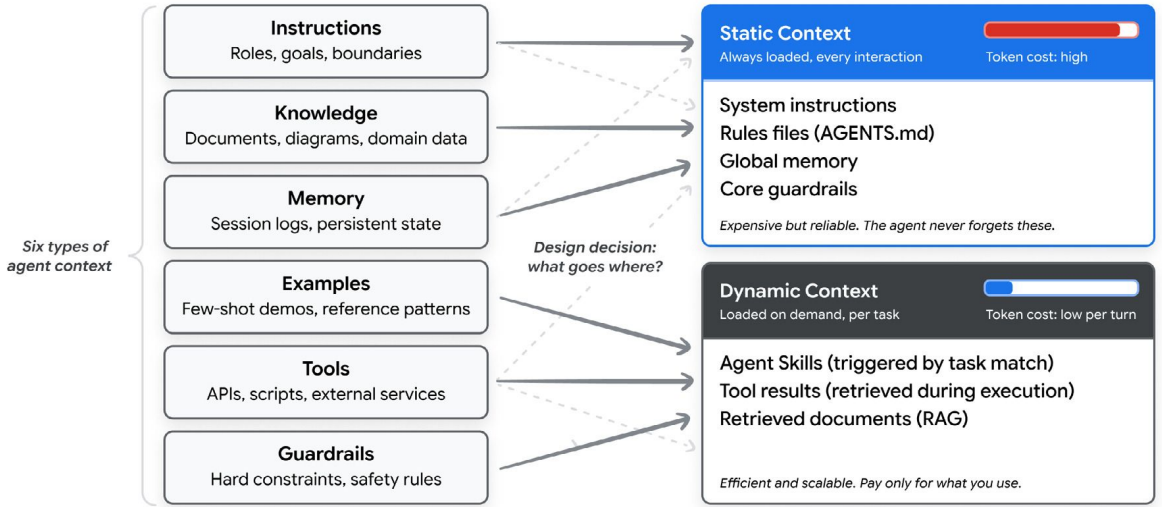
RECAP

How Each Phase of SDLC Actually Looks Like Today



RECAP

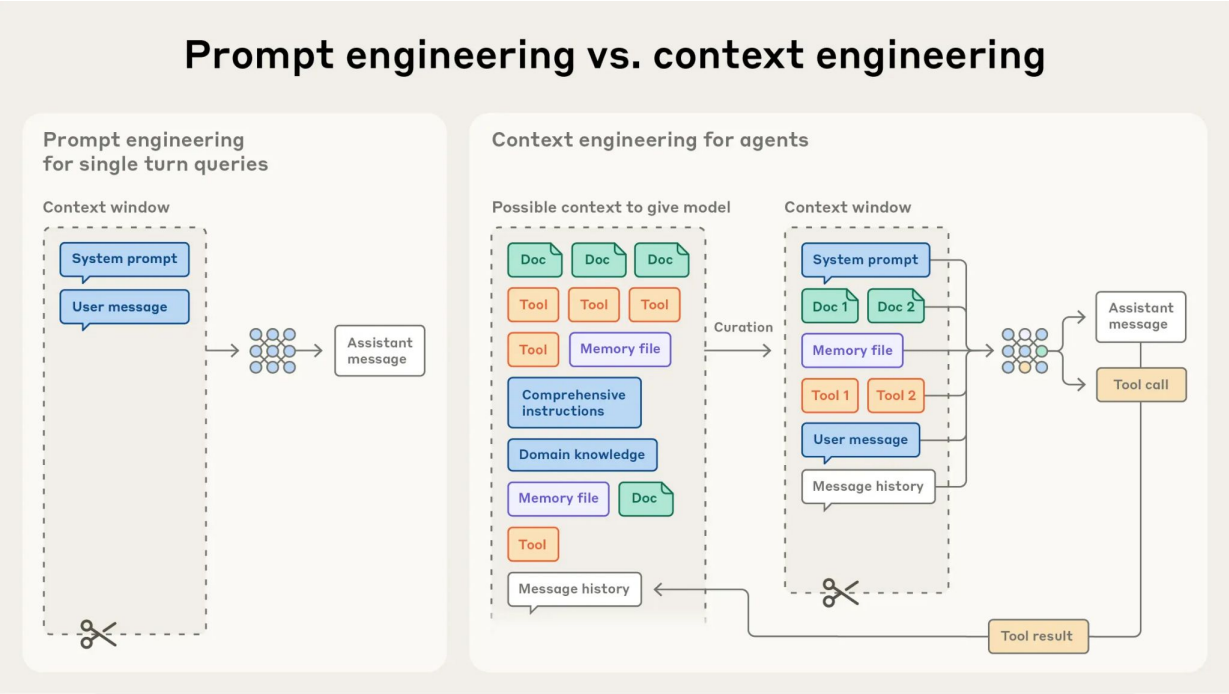
Context Engineering Is the Part That Decides Your Bill



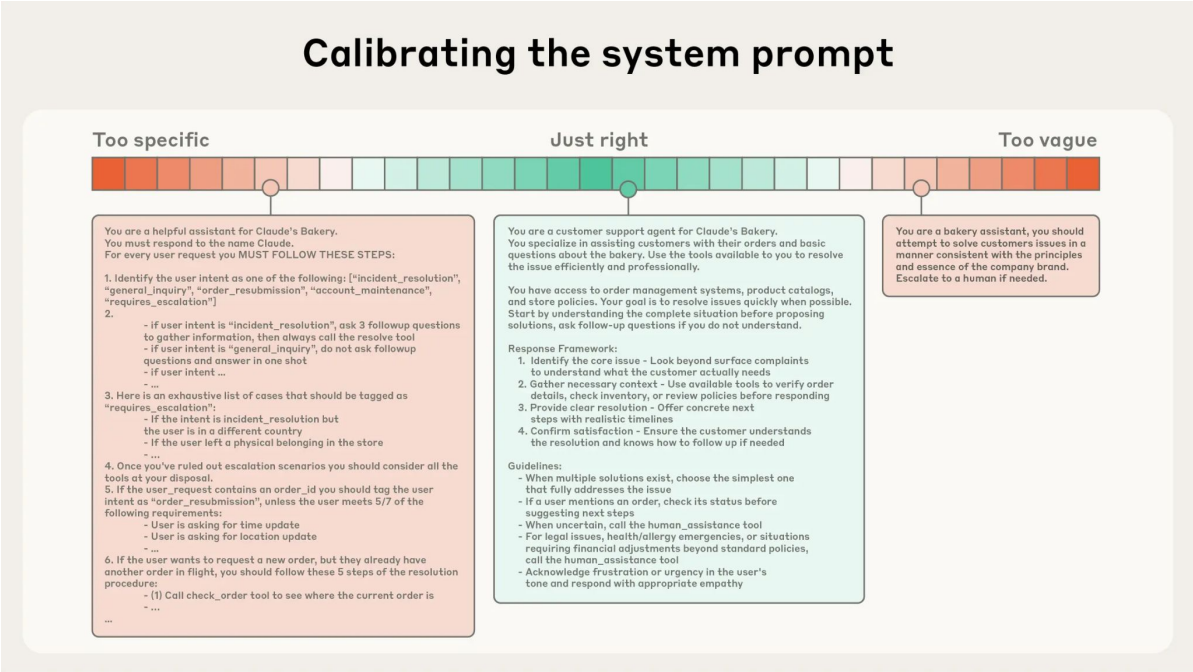
The best systems treat this as a first-class architectural decision, reviewed and versioned like code.

RECAP

Context Engineering



Calibrating the System Prompt



Thank You!

That’s a wrap on Day 4 — Agent Harness, Agent Orchestration & the Economies of AI Development 🙌

AI Assisted Development Program · Day 4